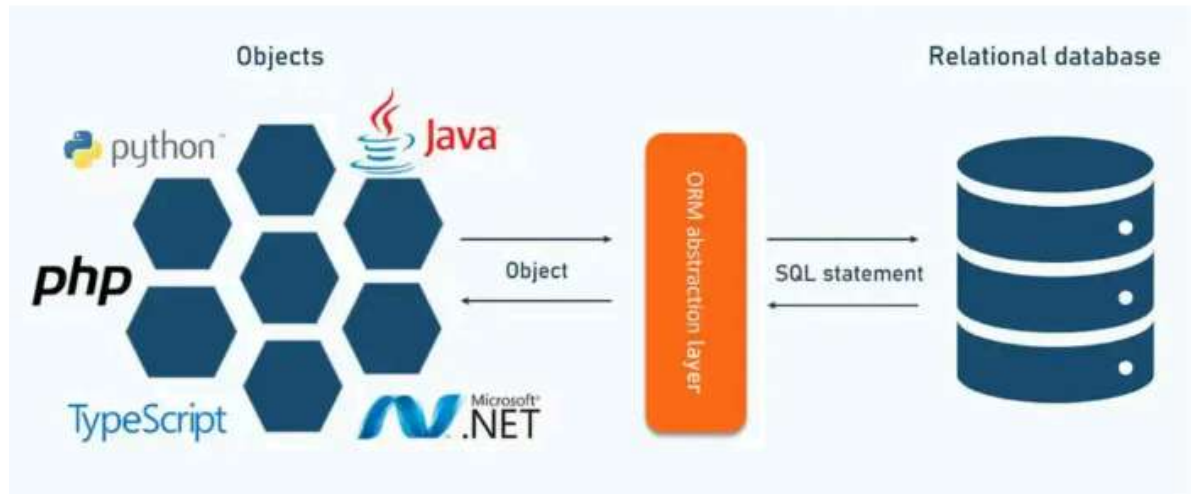


[Bài đọc] Tổng quan về ORM

1. ORM là gì?

- **ORM (Object-Relational Mapping)** là một kỹ thuật lập trình giúp ánh xạ (mapping) giữa các đối tượng trong ngôn ngữ lập trình hướng đối tượng (như Java) với các bảng trong cơ sở dữ liệu quan hệ (RDBMS)
- ORM cho phép lập trình viên thao tác với cơ sở dữ liệu thông qua các đối tượng mà không cần viết trực tiếp các câu lệnh SQL



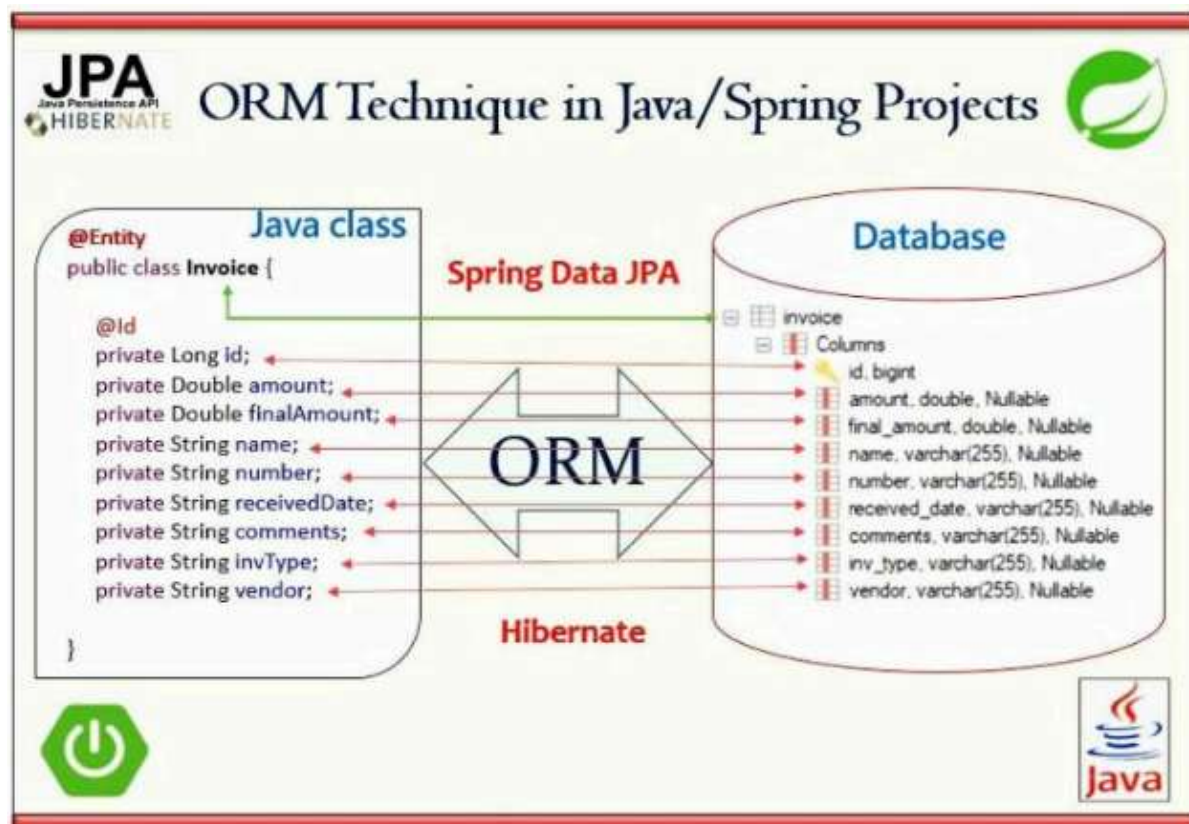
2. Tại sao cần ORM?

- **Giảm code SQL:** Bạn không cần viết tay các câu SQL như **SELECT**, **INSERT**, **UPDATE**, **DELETE**
- **Tăng tính trừu tượng và dễ bảo trì:** Làm việc với Java Objects thay vì các dòng SQL rời rạc
- **Đồng bộ dữ liệu và mô hình:** Giúp đảm bảo các lớp Java phản ánh chính xác cấu trúc của cơ sở dữ liệu

- Hỗ trợ tự động hóa: ORM hỗ trợ quản lý kết nối, giao dịch, cache...

3. Cách ORM hoạt động

- Mỗi **class** Java tương ứng với một **table** trong cơ sở dữ liệu
- Mỗi **instance** (đối tượng) tương ứng với một **record** (hàng dữ liệu)
- Các **field** trong class tương ứng với các **column** trong bảng
- ORM sẽ tự động chuyển đổi dữ liệu giữa Object và dữ liệu trong database (mapping dữ liệu)



4. Các ORM Framework phổ biến trong Java

Tên	Đặc điểm
Hibernate	Phổ biến nhất, hỗ trợ HQL (Hibernate Query Language), caching mạnh
JPA (Java Persistence API)	Chuẩn Java cho ORM, thường dùng với Hibernate làm hiện thực

Spring Data JPA	Dựa trên JPA, tích hợp sẵn với Spring Boot
-----------------	--

5. Ví dụ đơn giản với Hibernate (Annotation-based)

```

@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "username")
    private String username;

    @Column(name = "email")
    private String email;

    // getters và setters
}

```

- Annotation `@Entity` xác định đây là Entity class
- `@Table` ánh xạ với bảng `users`
- `@Id` xác định khóa chính
- ORM sẽ tự động ánh xạ các field sang column tương ứng

6. Ưu nhược điểm của ORM

Ưu điểm	Nhược điểm
Giảm code SQL	Học và hiểu framework mất thời gian
Dễ bảo trì, mở rộng	Có thể sinh SQL không tối ưu
Quản lý giao dịch, lazy loading	Debug query khó hơn
Tích hợp tốt với Spring	

7. Khi nào nên dùng ORM?

- Dự án cần làm việc nhiều với dữ liệu, nhiều entity, nhiều mối quan hệ
- Muốn quản lý giao dịch tốt
- Muốn tập trung phát triển nghiệp vụ, giảm lo về SQL

Danh sách các bài học

